

Interop



Simon Robinson

Software Developer

@TechieSimon | www.SimonRobinson.com



Overview



Interoperating with

- The native (unmanaged) Windows API
- Managed VB and F# code

Windows API Functions

- Enable more features in apps
- No longer confined to BCL capabilities

VB and F#

- Use code produced by teams working in those languages



Platform Check

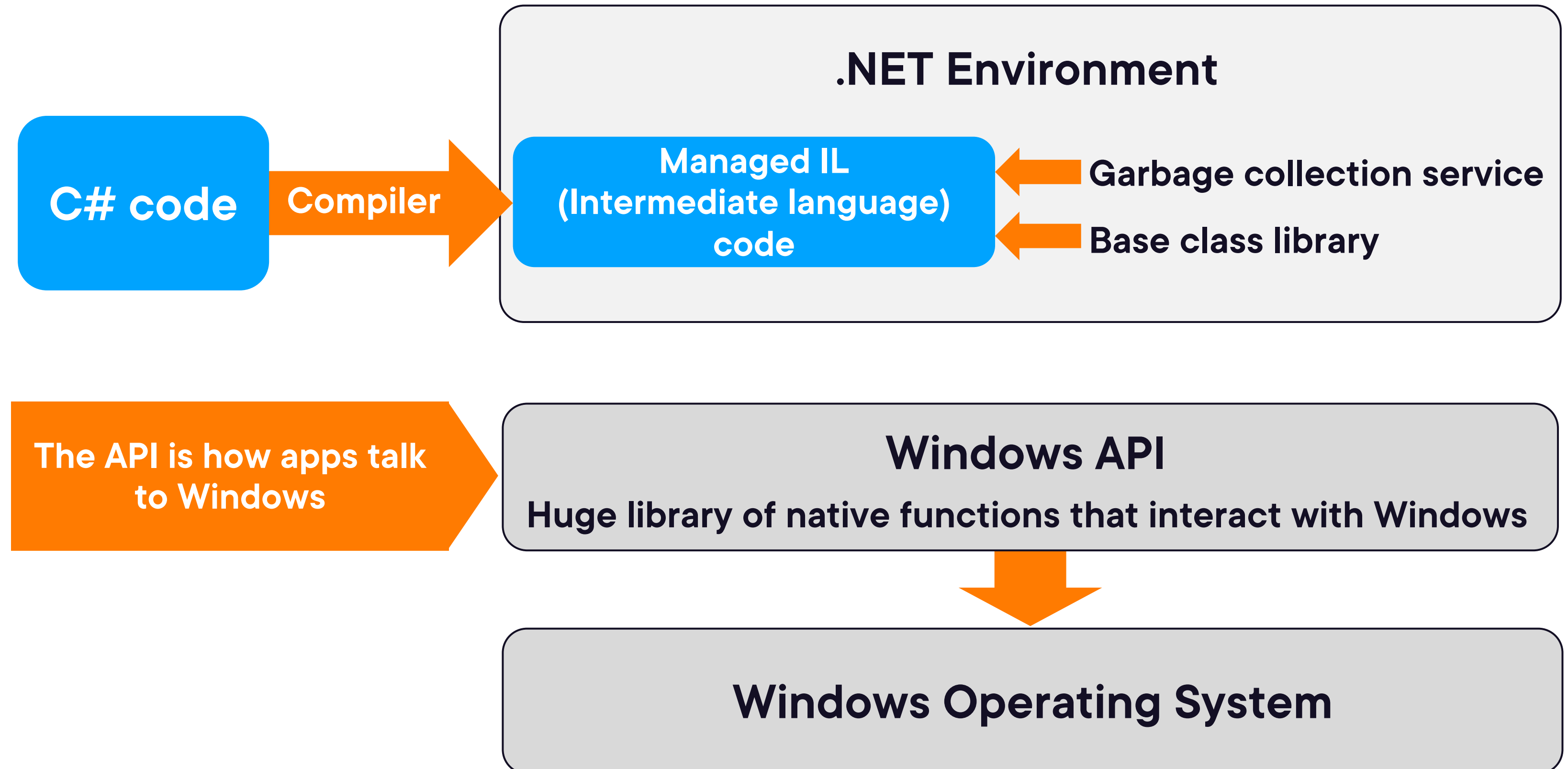


Windows API is applicable to

- MS Windows
- The principles of calling native code are cross-platform, but the code you'll see is Windows specific



Windows API



Windows API

Base class library
only provides subset of
Windows API features

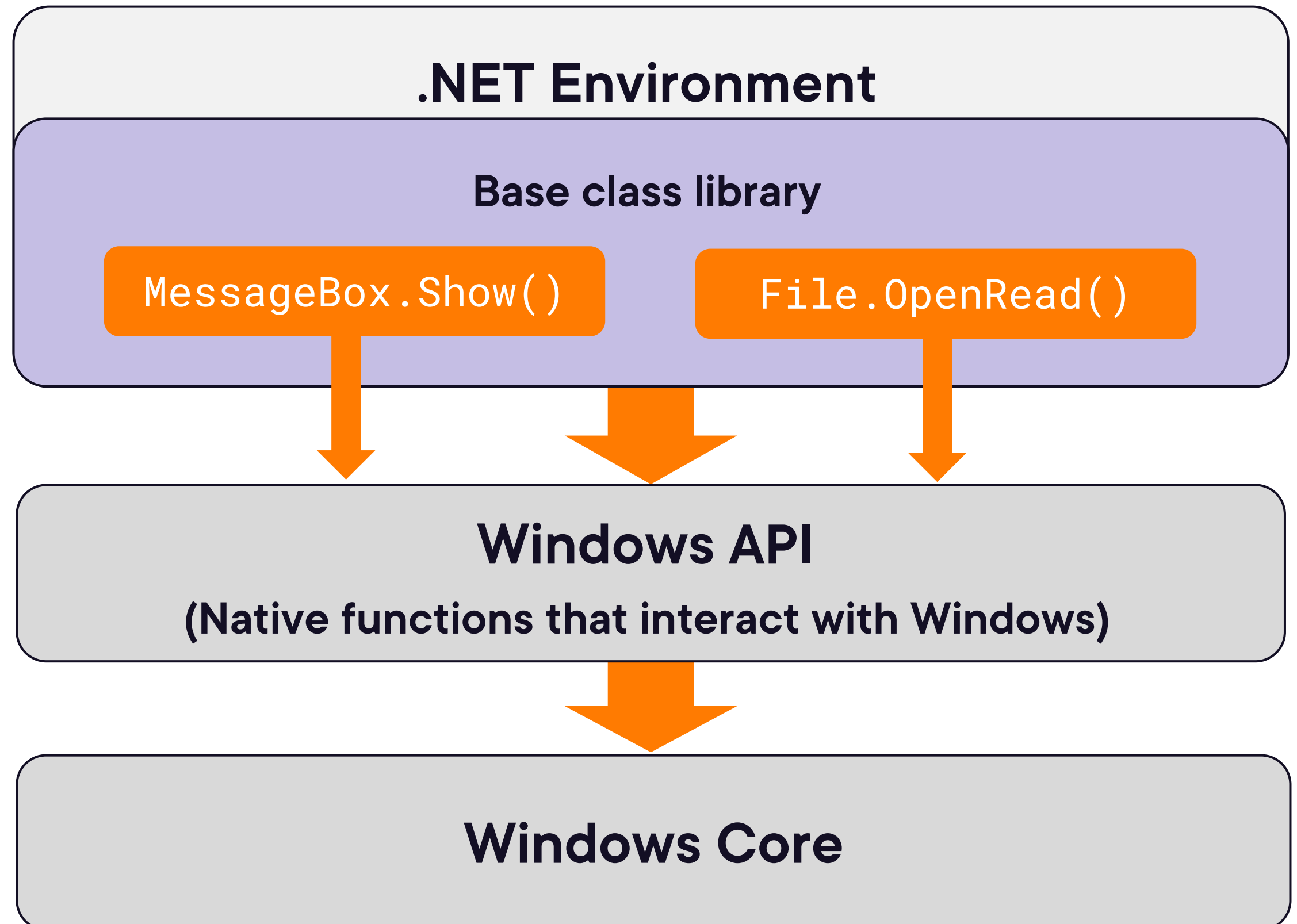
Call Windows API
directly
for everything else

For example:

Hardware access

Windows Shell
advanced features

DirectX graphics



Calling Windows API Functions



MessageBox function (winuser.h)

docs.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-messagebox

Microsoft

Docs

Documentation

Learn

Q&A

Code Samples

Shows

Events

Search

Sign in

Windows App Development

Explore

Development

Platforms

Resources

Dashboard

Filter by title

Dialog Boxes

> Commdlg.h

> Winuser.h

Overview

CreateDialogA macro

CreateDialogIndirectA macro

CreateDialogIndirectParamA function

CreateDialogIndirectParamW function

CreateDialogIndirectW macro

CreateDialogParamA function

CreateDialogParamW function

...

Win32

API

Dialog Boxes

Winuser.h

+

:

MessageBox function (winuser.h)

Article • 10/13/2021 • 7 minutes to read

👍🗨

Displays a modal dialog box that contains a system icon, a set of buttons, and a brief application-specific message, such as status or error information. The message box returns an integer value that indicates which button the user clicked.

Syntax

C++

Copy

In this article

Syntax

Parameters

Return value

Remarks

Show more

Demo



Calling MessageBox() API function

- P/Invoke layer
- Matching managed and unmanaged types



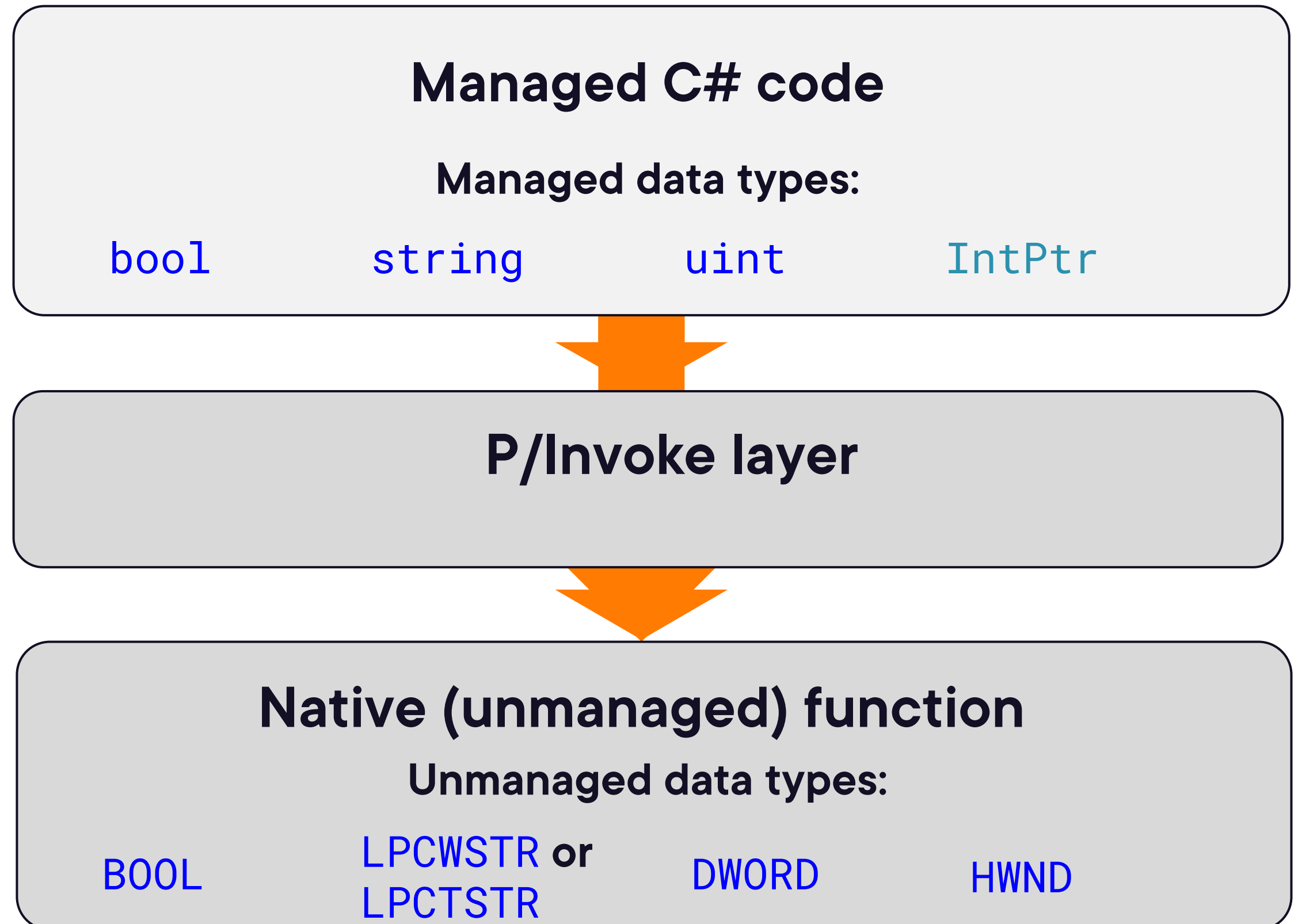
The Platform Invoke (P/Invoke) Layer

P/Invoke converts
between managed and
unmanaged types

This is called marshalling

You must still select
appropriate
managed types

You'll need to
recognise common
unmanaged types



Returning Data from an API Function



Displaying Monitors and Graphics Cards

EnumDisplayDevicesW function (x +

docs.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-enumdisplaydevicesw

... / Win32 / API / Windows GDI / Winuser.h /

⊕ ✎ ⋮

Filter by title

function

EnumDisplayDevicesW function

EnumDisplayMonitors function

EnumDisplaySettingsA function

EnumDisplaySettingsExA function

EnumDisplaySettingsExW function

EnumDisplaySettingsW function

EqualRect function

EnumDisplayDevicesW function (winuser.h)

Article • 10/13/2021 • 2 minutes to read

The **EnumDisplayDevices** function lets you obtain information about the display devices in the current session.

Syntax

C++ Copy

```
BOOL EnumDisplayDevicesW(  
    [in] LPCWSTR          lpDevice,  
    [in] DWORD            iDevNum,  
    [out] DISPLAY_DEVICEW lpDisplayDevice
```

In this article

- [Syntax](#)
- [Parameters](#)
- [Return value](#)
- [Remarks](#)

Show more ▾

Demo



Display monitors and graphics cards

- Using EnumDisplayDevicesW()
- Unicode vs. ANSI API methods
- Define struct to hold return values
- Custom marshalling



Strings

C# strings



**Always stored
as Unicode**

Native strings



**Can be stored
as Unicode**



**Can be stored
as ANSI**

Unicode

Much bigger character range
Required for globalisation

ANSI

Uses less memory
Better performance





Calling a Visual Basic Method



Demo



Outsource DisplayAdapters() method to VB

- This works just like calling a C# method





Calling an F# Method



Demo



Outsource DisplayAdapters() method to F#

- Works just like calling C# in theory
- But you need to be aware of type differences



Summary



Calling the Windows API

- Opens full Windows functionality to your C# apps
 - Including things not covered by the BCL
- Declare API methods as static extern
 - [DllImport]
- Work out appropriate types to marshal to unmanaged types
 - Use [MarshalAs] to guide marshalling
- Different coding techniques
 - Specifying data sizes
 - Calling a function multiple times



Summary



Calling other managed languages

- May be required if other teams use other languages
- VB and F#
 - Managed interop generally works out of the box
 - But beware that F# often uses different types